

BenefitPulse Engine — Technical Documentation

Technical Documentation | Standalone PDF edition

Product: BenefitPulse Engine Audience: Engineers, DevOps, technical reviewers Version: 1.0 · July 2026

1. Purpose of this document

This document describes how BenefitPulse is built, how to run it, how data and agents flow, and how to extend it. It assumes familiarity with web APIs, React, and cloud Postgres.

2. Technology stack

Layer	Choice	Role
Frontend	React 19, TypeScript, Vite, Tailwind CSS	SPA workspace UI
Routing / data	React Router, TanStack Query	Client navigation & caching
Backend	FastAPI (Python 3.11+)	REST API, orchestration
Agents	LangGraph + Google Gemini	Multi-step detection & assistant
RAG	ChromaDB + Gemini embeddings	Policy retrieval
Knowledge	Markdown under backend/knowledge_base/	Source of truth for policies
Data / Auth	Supabase (PostgreSQL + Auth + JWT)	Operational store
Local runtime	uvicorn, Vite dev server	Developer machines

Hard requirements (no offline demo mode)

The API will not start unless all of the following are set in backend/.env:

- GOOGLE_API_KEY
- SUPABASE_URL
- SUPABASE_ANON_KEY
- SUPABASE_SERVICE_ROLE_KEY
- SUPABASE_JWT_SECRET

There is no local JSON demo store. Auth is Supabase-only.

3. Repository layout (source)

```
card-benefit-activation-engine/  
├── frontend/                # React SPA  
│   ├── src/                 # Layout, AssistantChat, Simulate, UI kit  
│   └── components/
```

```

■      ■■■ pages/          # Overview, Coverage, Charges, Simulate, Cards, Claims, Auth
■      ■■■ context/       # AuthContext
■      ■■■ lib/           # api.ts, utils.ts
■■■ backend/
■      ■■■ app/
■      ■   ■■■ agents/    # pipeline.py (LangGraph), assistant.py
■      ■   ■■■ rag/       # vector_store.py (ChromaDB)
■      ■   ■■■ routers/   # auth, dashboard, benefits, claims, assistant
■      ■   ■■■ services/  # store, supabase_store, auth, supabase_client
■      ■   ■■■ models/    # Pydantic schemas
■      ■   ■■■ config.py  # Settings + validate_required()
■      ■   ■■■ main.py    # FastAPI entry
■      ■■■ knowledge_base/ # 6 policy markdown files
■      ■■■ scripts/       # seed_supabase.py, reindex_chroma.py, ...
■      ■■■ data/chroma/   # Local persistent vector index (gitignored)
■■■ supabase/schema.sql   # Postgres schema + RLS policies
■■■ README.md

```

4. Environment configuration

4.1 Backend (backend/ .env)

Variable	Required	Description
GOOGLE_API_KEY	Yes	Gemini API key
GEMINI_MODEL	No	Default gemini-2.5-flash
EMBEDDING_MODEL	No	Default models/gemini-embedding-001
GEMINI_FALLBACK_MODELS	No	Comma-separated fallbacks on 429/404
SUPABASE_URL	Yes	Project URL
SUPABASE_ANON_KEY	Yes	Browser/auth client key
SUPABASE_SERVICE_ROLE_KEY	Yes	Server-side writes (bypass RLS)
SUPABASE_JWT_SECRET	Yes	JWT validation
CORS_ORIGINS	No	Comma-separated origins
SUPABASE_SYNC	No	Audit agent runs (true default)
CHROMA_REINDEX	No	Force rebuild on boot

4.2 Frontend

Vite proxies API to `http://localhost:8000` by default. Optional frontend/.env:

```
VITE_API_URL=
```

Leave empty for proxy; set absolute URL only for split hosting.

5. Local runbook

```
# 1) Supabase: run supabase/schema.sql; enable Email auth
```

```
# 2) Backend
cd backend
```

```
python -m venv .venv
# Windows: .venv\Scripts\activate
pip install -r requirements.txt
# copy .env.example → .env and fill keys
python scripts/seed_supabase.py      # optional sample member + scenarios
python scripts/reindex_chroma.py     # Gemini embeddings → ChromaDB
uvicorn app.main:app --reload --port 8000

# 3) Frontend
cd frontend
npm install
npm run dev
```

Endpoint	URL
UI	http://127.0.0.1:5173
API docs	http://127.0.0.1:8000/docs
Health	http://127.0.0.1:8000/health
System status	http://127.0.0.1:8000/api/system/status

6. Data model (Supabase)

6.1 Core tables

Table	Purpose
profiles	User profile (1:1 with auth.users)
cards	Linked card products (card_name, card_type, last_four)
transactions	Posted charges
detected_benefits	Agent outcomes (eligible and outside coverage)
claims	Draft / submitted claim packets
documents	Uploaded receipts / proofs
agent_runs	Optional audit log of pipeline runs

6.2 detected_benefits.status values (prototype)

Status	Meaning
detected	Claim-eligible coverage identified
prefilled	Claim packet prefilled
not_eligible	Outside coverage (persisted after assessment)
submitted / approved / under_review	Claim lifecycle (as used)

Outside-coverage rows typically use:

- benefit_type = "Outside coverage"

- status = "not_eligible"
- max_coverage_amount = 0
- No claim draft created

6.3 RLS

Row Level Security is enabled so authenticated users only see their own rows. Server agents use the service role key for pipeline writes.

7. API surface (summary)

Method	Path	Description
POST	/api/auth/login	Supabase password login
POST	/api/auth/signup	Supabase signup + starter cards
GET	/api/dashboard	Profile, cards, txns, detections, stats
GET	/api/cards	Linked cards
GET	/api/transactions	Posted charges
GET	/api/benefits	Coverage ledger
GET	/api/benefits/{id}	Benefit detail (+ claim if eligible)
POST	/api/benefits/detect/{transaction_id}	Run pipeline on existing charge
POST	/api/benefits/inject	Create charge + optional detect
POST	/api/claims/{id}/submit	Submit claim
POST	/api/claims/{id}/documents	Upload document
POST	/api/assistant/chat	RAG-backed assistant
GET	/health	Liveness + stack flags
GET	/api/system/status	Architecture checklist
POST	/api/system/reindex-rag	Rebuild Chroma index

Dashboard stats includes:

- claim_eligible
- outside_coverage
- reviewed_charges
- potential_coverage
- submitted_claims

8. Agent pipeline

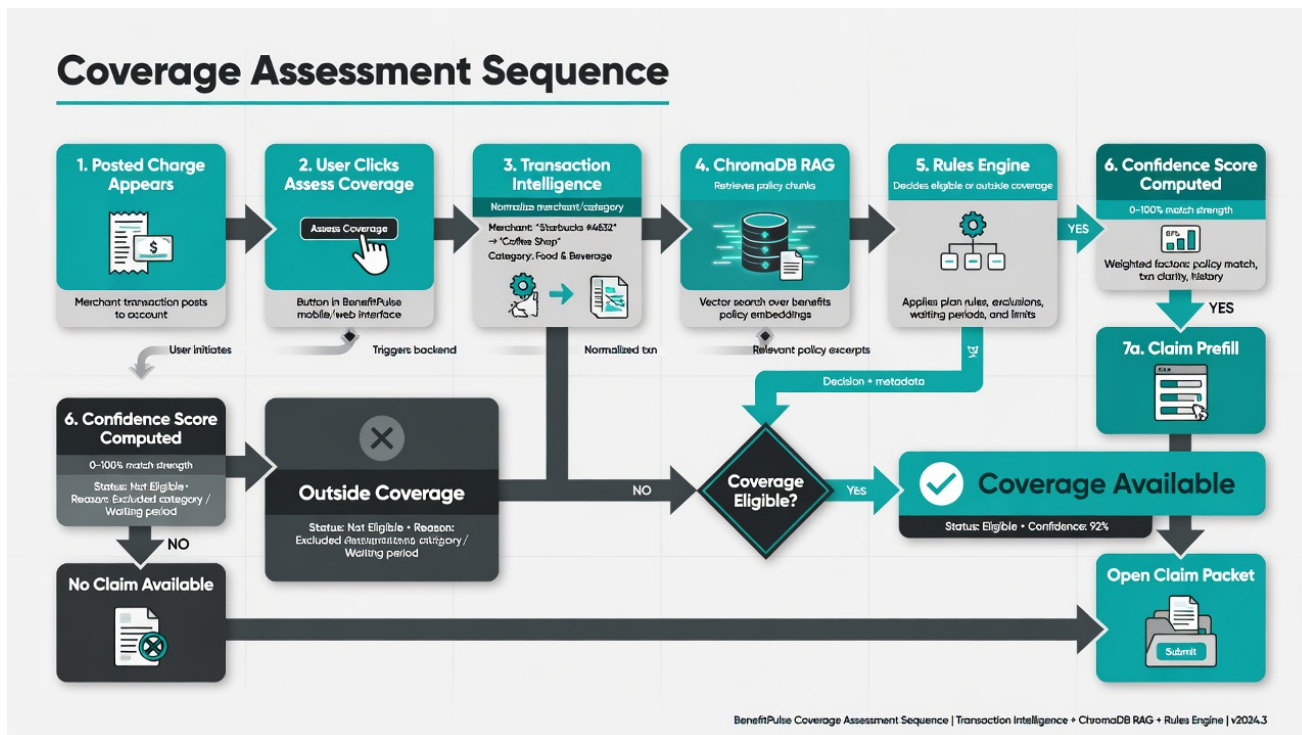


Figure: Coverage assessment sequence (conceptual).

8.1 Nodes (LangGraph / sequential fallback)

1. Transaction Intelligence — Normalize merchant, category, product type (Gemini when available; heuristics as backup for a single step).
2. Benefit Knowledge (RAG) — Query ChromaDB for policy chunks; propose benefit candidates.
3. Rules Engine — Deterministic eligibility (category, tier, window, exclusions).
4. Confidence Agent — Weighted score + breakdown.
5. Claim Prefill — Only if eligible; builds prefilled payload + missing documents.

8.2 Persistence after detect / inject

SupabaseStore.apply_pipeline_result:

- Always upserts a detected_benefits row for the transaction.
- Eligible: status detected or prefilled; creates/updates draft claim.
- Not eligible: status not_eligible, type Outside coverage; no claim draft.
- Updates transaction merchant/category from intelligence when present.

8.3 Knowledge base files

purchase_protection_guide.md
 return_protection_policy.md
 travel_delay_insurance.md
 extended_warranty.md
 general_exclusions.md
 faq_benefits.md

Reindex:

```
python scripts/reindex_chroma.py
# or POST /api/system/reindex-rag
```

9. Frontend architecture

9.1 Pages / routes

Route	Page	Role
/login, /signup	Auth	Supabase credentials
/dashboard	Overview	Stats + recent assessments
/benefits	Coverage ledger	All detections
/transactions	Posted charges	Assess / re-assess
/simulate	Simulate	Live inject + pipeline
/cards	Cards	Wallet UI
/profile	Profile	Account
/claims/:benefitId	Claim packet	Prefill, upload, submit

9.2 Key client helpers

- `api.ts` — typed REST client + bearer token
- `utils.ts` — `isClaimEligible`, `coverageStatusLabel`, `confidenceLabel`, money/date format
- `AuthContext` — session in local storage; attaches `Authorization` header

9.3 UI states for charges

Assessment state	Row treatment
Pending	Neutral
Claim-eligible	Emerald highlight
Outside coverage	Stone / grey highlight

10. Seed script

`python scripts/seed_supabase.py`

Creates / updates sample member `demo@amex.com` / `demo1234` with:

- The Platinum Card® + American Express® Gold Card
- 10 transactions spanning high / medium / low confidence and outside coverage
- Draft claims only for claim-eligible detections
- Clears prior seed transactions/detections/claims for that user before insert

11. Security notes

1. Never commit backend/ .env.
2. Service role key is server-only.
3. Prefer short-lived JWT validation with `SUPABASE_JWT_SECRET`.
4. Prototype CORS may be permissive for local demos — tighten for any shared deploy.

5. Do not store full PAN; only last four of product records.
6. Gemini receives merchant/description text — avoid PII beyond what is needed for demo.

12. Observability & ops

Check	Healthy signal
/health	status: ok, gemini_configured: true, data_backend: supabase
/api/system/status	rag.backend: chromadb, supabase.tables_ok: true
Logs	Startup line: data=supabase, gemini=...

Common failures:

Symptom	Fix
Startup RuntimeError missing credentials	Fill .env
No module named supabase	pip install -r requirements.txt
rag.backend = keyword	Install chromadb; set Gemini; reindex
Login 401 after seed	Disable email confirm; re-run seed
Gemini 429	Switch model / wait; rules may still finish a step

13. Extension points

1. New benefit type — Add policy markdown, extend ELIGIBLE_CATEGORIES / rules, reindex.
2. New UI workspace — Add route + nav item in Layout .tsx.
3. Webhooks — Replace inject/simulate with bank/issuer transaction feed.
4. Human review queue — Filter low-confidence detected for ops.
5. Production auth — Move SPA to Supabase client SDK with PKCE; keep service role server-side only.

14. Test checklist (manual)

- [] Backend refuses start without keys
- [] Login seed user works
- [] Overview shows claim-eligible + outside coverage counts
- [] Charges highlight both outcome types
- [] Assess on food charge → outside coverage persisted
- [] Assess on electronics → claim-eligible + claim packet
- [] Simulate laptop + food scenarios
- [] Assistant answers with policy grounding
- [] Upload document + submit claim

15. Versioning

Prototype version 1.0. Document updates should track major UX or data-model changes (especially status enums for detections).